

Fedora Server

Instalação segura com LVM sobre LUKS2, hardening de SSH e detecção de brute force

Grupo 5 · Fedora Server 44 · Script: ssh-sentinel.sh

Integrantes: Bruno Tomé Duarte · Gabriel Zobolli Carnevalli · Matheus Pascoal Craveiro · Heitor Fonseca Amorim dos Santos

Professor: Fábio Silva Pires de Oliveira · Turma: 1TDPCG · Entrega: 16/08/2026

30 minutos, seis blocos cronometrados

4 min

Abertura e distribuição

Quem mantém o Fedora, ciclo de vida, quando escolher

5 min

Esquema de particionamento

Diagrama e justificativa de cada decisão

7 min

Instalação com LVM e LUKS

Pontos críticos da instalação, ao vivo/acelerado

6 min

Serviço SSH endurecido

Acesso por chave funcionando e tentativa bloqueada

5 min

Script em shell

Arquitetura e execução ao vivo com saída real

3 min

Cinco perguntas

Aplicação do questionário à turma

Quem mantém o Fedora

1

Projeto comunitário, patrocinado pela Red Hat

Fundado em nov/2003, quando a Red Hat separou o antigo Red Hat Linux em RHEL (produto comercial) e Fedora (comunitário, uso livre).

2

Milhares de colaboradores independentes

A Red Hat (hoje subsidiária da IBM) é a patrocinadora principal, mas colaboradores não afiliados formam a maioria da comunidade ativa.

3

Fedora Server é a edição deste trabalho

Instalação mínima, sem ambiente gráfico — o perfil exigido pelo enunciado ("Server sem GUI / Minimal Install").

Fedora é o upstream do RHEL



"O Fedora continua sendo o primeiro upstream do RHEL — é dali que toda versão do RHEL historicamente se origina."

Consequência prática: recursos e políticas de segurança (SELinux, firewalld, OpenSSH) chegam primeiro ao Fedora e, meses ou anos depois, de forma mais conservadora, ao RHEL.

Ciclo de lançamento e de manutenção

~6

meses entre lançamentos
(duas versões por ano)

~13

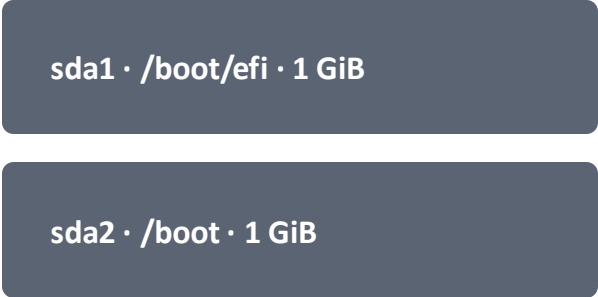
meses de suporte por versão
(até 4 semanas após a versão X+2)

44

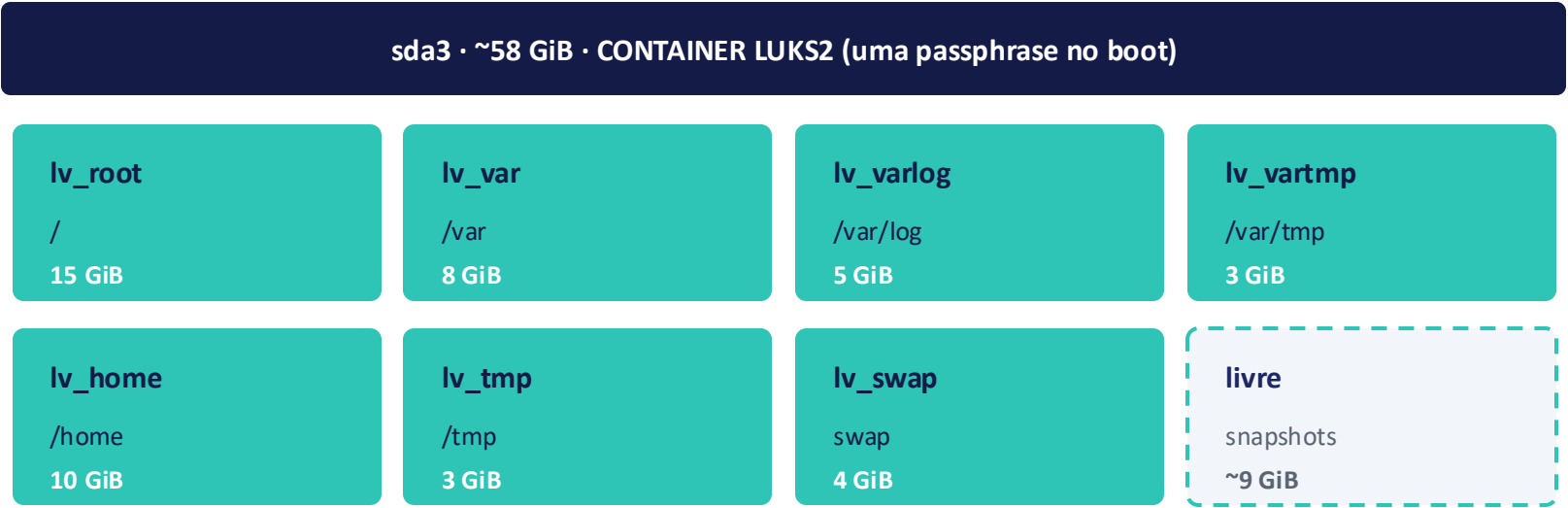
versão instalada pelo grupo
(Fedora-Server-dvd-x86_64-44)

Quando escolher Fedora Server: laboratório, testes, aprendizado do ecossistema Red Hat e ambientes que toleram atualização contínua — não é a escolha para produção de longo prazo sem um processo maduro de upgrade.

LVM sobre LUKS2 — diagrama de referência



Fora do LUKS — o GRUB precisa ler o kernel antes de existir chave



VG: vg_sistema (PV = /dev/mapper/cryptlvm)

Por que o esquema foi desenhado assim

1

Um único container LUKS para todo o VG

Uma só passphrase no boot; qualquer LV novo — inclusive vindo do disco secundário — já nasce criptografado.

2

/boot e /boot/efi fora da criptografia

Risco residual aceito e documentado: quem tem acesso físico ao disco pode ler o /boot. Mitigação completa (Secure Boot) fica fora do escopo.

3

~9 GiB livres no VG não são desperdício

É requisito funcional do LVM: sem espaço livre não há snapshot nem como socorrer um volume que encheu. VG a 100% é problema, não otimização.

Opções de montagem restritivas

Ponto de montagem	Opções aplicadas	O que bloqueia
/tmp	nodev, nosuid, noexec	Payload executado em diretório mundialmente gravável
/var/tmp	nodev, nosuid, noexec	Mesmo vetor, na área temporária de serviços
/var/log	nodev, nosuid, noexec	Área de log usada como ponto de staging de ataque
/home	nodev, nosuid	Binário SUID criado dentro do próprio diretório do usuário
/boot	(sem noexec)	GRUB precisa executar o kernel a partir daqui

Conflito documentado: noexec em /var/tmp pode afetar scripts pós-instalação do dnf — testado com "dnf check-update" (sem impacto); decisão do grupo: manter noexec, remontando com exec só em caso de falha real de atualização.

LUKS2: por que este formato

1

Metadados em JSON

auditáveis e com redundância — reparo automático a partir de cópia

2

Até 32 key slots

contra 8 do LUKS1 — múltiplas passphrases/chaves para o mesmo volume

3

Reencrytação online

troca de chave ou algoritmo sem desmontar o sistema de arquivos

4

Limitação conhecida

protege só com a máquina desligada — não substitui SELinux nem controle de acesso do SO

Ambiente de laboratório

Item	Configuração
Hipervisor	Oracle VirtualBox
Firmware	UEFI habilitado (nunca BIOS legado)
Disco principal / secundário	60 GB (VDI) / 20 GB (adicionado após a instalação)
Memória / vCPU	4096 MB / 2 vCPUs
Rede	Adaptador 1: NAT · Adaptador 2: Host-Only (192.168.56.0/24)
ISO	Fedora-Server-dvd-x86_64-44-1.7.iso — hash SHA-256 verificado antes de instalar

Pontos críticos da instalação (Anaconda)

1

Installation Destination → Custom → LVM + Encrypt my data

Sem particionamento automático; disco de 60 GB selecionado manualmente.

2

Criação manual das 9 partições/volumes na ordem do diagrama

/boot/efi e /boot primeiro (fora do LUKS), depois os volumes lógicos do vg_sistema.

3

Renomear o Volume Group para vg_sistema e marcar Encrypt

Antes de clicar em Done — é nesse passo que o Anaconda associa o LUKS ao VG.

4

Passphrase do LUKS anotada em dois lugares diferentes

Crítico: sem ela e sem backup do header, não existe recuperação possível.

Verificações pós-instalação (evidências)

1

Sistema e SELinux

`cat /etc/os-release · uname -r · getenforce (Enforcing) · sestatus`

2

Disco, LUKS e LVM

`lsblk -f · cryptsetup luksDump /dev/sda3 · pvs / vgs / lvs`

3

Pontos de montagem

`findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS · cat /etc/fstab · cat /etc/crypttab`

Snapshot 1 tirado logo após a instalação limpa — antes de qualquer configuração de SSH ou usuários.

Ciclo de vida do LVM: disco secundário de 20 GB



Sem desligar a VM: o /home cresce de ~10 GiB para ~20 GiB a quente — demonstração ao vivo (ou vídeo acelerado) neste momento da apresentação.

Verificação final: `lvs` e `df -h /home`

Configuração aplicada em sshd_config

Diretiva	Valor	Objetivo
PasswordAuthentication	no	Elimina força bruta contra senha — só chave ed25519 autentica
PermitRootLogin	no	Acesso sempre nominal; privilégio elevado só via sudo
Port	2222	Porta não padrão — reduz ruído de varredura automatizada
AllowGroups	ssh-users	Lista de permissão explícita para o grupo dedicado
MaxAuthTries / LoginGraceTime	3 / 30	Limita tentativas por conexão e tempo de autenticação
ClientAliveInterval / CountMax	300 / 2	Encerra sessões ociosas automaticamente

SELinux, firewalld e política de criptografia

1

Rótulo SELinux da nova porta

`sudo semanage port -a -t ssh_port_t -p tcp 2222 — evidenciado em evidencia-selinux-porta.txt (ssh_port_t: 2222, 22).`

2

firewalld liberando só a porta nova

`firewall-cmd --add-port=2222/tcp + rich rule com rate limit ("10/m").`

3

Cifras e MACs legados removidos

`sudo update-crypto-policies --set DEFAULT — remove algoritmos fracos do conjunto negociável.`

4

Banner legal em /etc/issue.net

Aviso de acesso restrito e de que sessões são registradas, citando o Art. 154-A do Código Penal.

Evidência: acesso por chave funcionando

```
Sep 14 20:59:41 sshd-session[8630]: Accepted publickey for heitor from 100.127.64.96 port 48583 ssh2: ED25519  
SHA256:iMo2er66rNI...  
Sep 14 20:59:41 sshd-session[8630]: pam_unix(sshd:session): session opened for user heitor(uid=1003) by  
heitor(uid=0)  
Sep 14 20:59:23 sshd[8625]: Server listening on 0.0.0.0 port 2222.
```

`ssh -i ~/.ssh/id_ed25519 -p 2222 heitor@100.98.214.73` - Entrar na VM

`sudo journalctl -u sshd --since "-2min"` - Mostrar a log

journalctl -u sshd — três integrantes (heitor, pascoal, bruno) autenticados por chave ed25519, acesso remoto via Tailscale

Evidência: tentativa bloqueada + a regra de ouro

```
Sep 14 21:02:02 sshd-session[8754]: Invalid user usuario_fantasma from 100.127.64.96 port 48632  
Sep 14 21:02:02 sshd-session[8754]: Connection reset by invalid user usuario_fantasma 100.127.64.96 port 48632 [preauth]
```

`ssh -p 2222 -o PubkeyAuthentication=no usuario_fantasma@100.98.214.73 -`
Tentativas sem autenticação por chave

A regra de ouro seguida pelo grupo

Antes de reiniciar o sshd: validamos a sintaxe com `sshd -t` e mantivemos uma sessão SSH aberta em paralelo, testando a nova configuração numa segunda sessão antes de fechar a original.

Em VM, o custo do erro é um snapshot; em produção, é um chamado de madrugada.

Requisitos comuns aos 6 scripts do trabalho

set -euo pipefail

Mínimo 5 funções

getopts / case, -h --help

Checa EUID e command -v

Log com timestamp INFO/WARN/ERROR

Códigos de saída 0-3

Idempotente

Sem segredo no código

trap + mktemp

Aprovado no shellcheck

Reversível

Comentários em português

ssh-sentinel.sh — arquitetura

1. Coleta
journalctl (unit sshd)

2. Parsing
IP + usuário da falha

3. Agregação
contagem por IP/janela

4. Decisão
limiar × allowlist

5. Ação
rich rule + TTL no firewalld

6. Relatório
top 10 IPs / usuários

Allowlist obrigatória: a rede de gerência e o IP do administrador nunca são bloqueados, mesmo ultrapassando o limiar.

Autor: Matheus Pascoal · padrões reais do script: limiar 5 falhas, TTL 1h, janela "1 hour ago" · bloqueios via /etc/ssh-sentinel/excecoes.conf

ssh-sentinel.sh vs. fail2ban — e execução ao vivo

Aspecto	ssh-sentinel.sh (grupo)	fail2ban (referência de mercado)
Execução	Sob demanda / agendada (timer)	Daemon persistente, quase tempo real (inotify)
Fonte	journald (journalctl -u sshd)	Log tradicional ou journald, conforme backend
Ação	Rich rule do firewalld, TTL no próprio script	Módulo de ação (iptables/nftables/firewalld/ufw)
Reincidência	Extensão futura possível	Jail "recidive" com bans progressivos

Execução ao vivo nesta seção: `./ssh-sentinel.sh -h` · `shellcheck ssh-sentinel.sh` · `sudo ./ssh-sentinel.sh --dry-run` · saída real de uma execução completa contra a simulação de brute force em rede isolada.

MÚLTIPLA ESCOLHA

No ecossistema Red Hat, qual é a posição do Fedora Linux em relação ao RHEL?

- a) O Fedora é um clone binário do RHEL, lançado depois dele, sem relação de dependência de código
- b) O Fedora é o upstream mais próximo do usuário final: recursos são testados nele primeiro e, depois, chegam ao RHEL via CentOS Stream
- c) O RHEL é que serve de upstream para o Fedora — a Red Hat testa recursos no RHEL antes de portá-los para o Fedora
- d) Fedora e RHEL são projetos totalmente independentes, mantidos por empresas diferentes, sem nenhuma relação técnica

b

Alternativa correta

O Fedora reúne código de dezenas de projetos upstream para montar o Fedora Linux; periodicamente, a Red Hat parte de um lançamento do Fedora para basear uma versão do RHEL, hoje com o CentOS Stream como estágio intermediário. (a) descreve na verdade os clones binários como Rocky/AlmaLinux; (c) inverte o fluxo real; (d) ignora que a própria Red Hat patrocina o Fedora.

MÚLTIPLA ESCOLHA

Por que /boot/efi e /boot ficam fora do container LUKS no esquema deste trabalho?

- a)** Porque partições pequenas (abaixo de 2 GiB) não podem ser criptografadas pelo cryptsetup
- b)** Porque o GRUB precisa conseguir ler o kernel e a initramfs antes de existir qualquer chave de descriptografia disponível
- c)** Porque o firmware UEFI bloqueia automaticamente a criptografia de qualquer partição de boot
- d)** Porque deixar o /boot fora do LUKS elimina totalmente o risco de acesso físico ao disco



Alternativa correta

É o próprio GRUB (ou um estágio anterior) que solicita a passphrase do LUKS para desbloquear o restante do disco — por isso precisa acessar o kernel antes de qualquer chave existir. Isso deixa um risco residual conhecido: acesso físico ao disco expõe o /boot. (a) e (c) são tecnicamente falsas; (d) contradiz o próprio risco residual apresentado.

MÚLTIPLA ESCOLHA

No script `ssh-sentinel.sh`, qual é a função da `allowlist` (lista de exceções)?

- a)** Listar os usuários que têm permissão para usar senha em vez de chave pública
- b)** Listar os IPs que nunca devem ser bloqueados pelo script, mesmo ultrapassando o limiar de falhas configurado
- c)** Listar os algoritmos de criptografia permitidos pela política do `update-crypto-policies`
- d)** Listar as portas que o `firewalld` deve manter sempre abertas, independentemente do script



Alternativa correta

A allowlist impede que o próprio script bloqueie a rede de gerência da VM ou o IP do administrador em teste — sem ela, o `ssh-sentinel.sh` poderia causar a negação de serviço que deveria prevenir. As demais alternativas confundem esse conceito com mecanismos independentes (autenticação por chave, política de criptografia e regras do `firewalld`).

DISSERTATIVA CURTA

O trabalho aplica desconto de 10 pontos caso o SELinux esteja desligado — o maior desconto pontual da rubrica. Por que o SELinux é tratado como controle tão central, mesmo com o disco já protegido por LUKS e o SSH já endurecido?

Resposta em até 3 linhas.

LUKS e SSH endurecido protegem, respectivamente, os dados em repouso e o acesso remoto — mas nenhum dos dois controla o que um processo já em execução (mesmo como root) pode fazer dentro do sistema ligado. O SELinux aplica controle de acesso obrigatório baseado em rótulos, cobrindo exatamente a lacuna que LUKS (só protege desligado) e o SSH (só controla a porta de entrada) deixam aberta.

DISSERTATIVA CURTA

O grupo documentou um conflito entre a opção noexec em /var/tmp e o funcionamento do dnf. Explique o conflito e a decisão tomada.

Resposta em até 3 linhas.

O dnf usa /var/tmp para descompactar pacotes durante atualizações; scripts pós-instalação executados a partir dali podem falhar com noexec ativo. Testado com "dnf check-update", que funcionou normalmente. Decisão do grupo: manter noexec por padrão — o ganho de segurança supera o risco — reservando a remontagem temporária com exec só para uma atualização que realmente falhe por esse motivo.